Week 9: Constraint Satisfaction Problem (CSP)

CLO - 3: Design basic AI-based solutions for real-world problems using appropriate techniques such as reasoning, tracking, or robot localization.

Submission Guidelines:

1. Each task must properly show the output. DO NOT Clear the outputs after running.
2. **Download the .ipynb file** from Jupyter Notebook.
3. **Upload the .ipynb** to GCR.
4. **Failure to follow the submission guidelines will result in a deduction of 50% of the obtained marks.**

Task 1: Smart Campus Exam Scheduler Using Backtracking

Your university needs to schedule examinations for six courses. Some courses have overlapping students, so their examinations cannot take place in the same time slot. Your task is to develop a CSP-based scheduler that assigns time slots while satisfying all examination-conflict constraints.

Given data

The courses are: AI, DB, CN, SE, ML, DS

The available time slots are: Mon-9, Mon-11, Tue-9, Tue-11

Initially, every course can use any of the four time slots.

The following pairs of courses have overlapping students and therefore cannot be scheduled simultaneously:

- AI and DB
- AI and CN
- AI and DS
- DB and SE
- DB and ML
- CN and ML
- SE and DS
- ML and DS

Requirements

1. Represent each course as a CSP variable.

2. Define the domain of each course using the available time slots.
3. Represent the examination conflicts using a constraint graph.
4. Implement binary constraints that prevent conflicting courses from receiving the same time slot.
5. Implement a backtracking search algorithm that assigns one course at a time.
6. Check whether a candidate assignment is consistent with all previously assigned courses.
7. If an assignment leads to a dead end, undo it and try another available time slot.
8. Display the final timetable if a solution exists; otherwise, display an appropriate failure message.

Expected output

Your program should display a timetable similar in structure to the following:

Course	Assigned time slot
AI	Mon-9
DB	Mon-11
CN	Mon-11
SE	Mon-9
ML	Tue-11
DS	Tue-9

The exact solution may differ, provided all constraints are satisfied.

Learning outcome: *Students will learn how to formulate a real-world scheduling problem as a CSP and implement basic backtracking search.*

Task 2: Intelligent Course Scheduling Using MRV, Degree Heuristic, and LCV

Your scheduler from Task 1 works correctly, but it selects variables and values in a fixed order. For larger scheduling problems, this may result in unnecessary search.

Improve the existing solver by implementing variable- and value-selection heuristics.

Requirements

1. Extend your Task 1 implementation rather than writing a new solver from scratch.

2. Implement the Minimum Remaining Values (MRV) heuristic to select the unassigned course with the fewest currently legal time slots.
3. If multiple courses have the same MRV score, use the Degree Heuristic to select the course that constrains the largest number of other unassigned courses.
4. Implement the Least Constraining Value (LCV) heuristic to try the time slot that eliminates the fewest available choices for neighboring courses.
5. Integrate these heuristics into the existing backtracking algorithm.
6. Count the number of candidate assignments attempted during search.
7. Run the scheduler using:
    - Basic backtracking with the original variable and value order.
    - Backtracking with MRV, Degree, and LCV.
8. Display the resulting timetable and number of candidate assignments attempted for both approaches.

Analysis questions

- Why might MRV detect failure earlier than selecting the first unassigned course?
- Why is the Degree Heuristic useful when two courses have the same MRV score?
- How does LCV help preserve options for other courses?
- Did the heuristics reduce the number of candidate assignments in your experiment? Explain your observation.

Learning outcome: *Students will understand how search heuristics influence efficiency without changing the definition of a valid solution.*

Important: The heuristics do not guarantee that every CSP will be solved with fewer search operations. Report the results of your actual implementation rather than assuming the optimized version must always be faster.

Task 3: Smart Scheduler with Forward Checking

Your university scheduler sometimes assigns a time slot to one course that leaves another course with no legal time slot. Basic backtracking may discover this problem only when it attempts to assign the affected course.

Improve the scheduler by implementing Forward Checking.

Requirements

1. Continue using the CSP representation and backtracking framework from **Tasks 1 and 2**.
2. Maintain a separate, modifiable copy of the domains during search.
3. Whenever a course receives a time slot, inspect its unassigned neighboring courses.
4. Remove the assigned time slot from the domains of neighboring courses that conflict with it.
5. If any unassigned course has an empty domain, immediately reject the current branch and backtrack.
6. Record the domain values removed during each trial or use a domain snapshot.
7. Restore the domains correctly whenever a trial fails. A failed branch must not affect subsequent branches.
8. Run the scheduler using basic backtracking and forward checking.
9. Display the solution, the number of candidate assignments attempted, and the number of domain values removed during propagation.

Required demonstration

Add a debug log showing at least one assignment and its effect on neighboring domains. For example, when a course is assigned Mon-9, show which conflicting courses lose that time slot.

If a branch causes a domain to become empty, display the affected course and explain why the algorithm backtracks.

Analysis questions

- How does forward checking differ from ordinary consistency checking?
- Why must removed domain values be restored after a failed branch?
- Can forward checking detect every inconsistency before a complete assignment is attempted? Explain.

Learning outcome: *Students will learn how domain pruning reduces unnecessary exploration and why correct state restoration is essential in recursive search.*

Task 4: Network Constraint Analyzer Using AC-3 and MAC

A university is configuring wireless channels for a group of nearby access points. Access points that interfere with each other cannot use the same channel.

Your task is to implement two constraint-propagation techniques: AC-3 and Maintaining Arc Consistency (MAC).

Given data

The access points are:

A, B, C, D, E

Their initial channel domains are:

Access point	Available channels
A	1
B	12
C	123
D	23
E	3

The interference relationships are:

- A interferes with B.
- B interferes with C.
- C interferes with D.
- D interferes with E.

For every interfering pair, the assigned channels must be different.

Part A: Implement AC-3

1. Represent the access points as variables and their available channels as domains.
2. Define the binary constraint for interfering access points.
3. Create a queue containing the directed arcs associated with the constraints.
4. Implement the revise(X, Y) operation to remove values from the domain of X that have no compatible supporting value in the domain of Y.
5. Implement the AC-3 queue-processing algorithm.
6. Whenever a domain changes, add the relevant neighboring arcs back to the queue.
7. Return failure if any domain becomes empty; otherwise, report that arc consistency has been achieved.

8. Display the initial and final domains.

Part B: Implement MAC

Extend your backtracking solver to use MAC.

1. Select an unassigned variable and try one of its available channels.
2. Assign the channel and restrict the variable's domain to that value.
3. Run AC-3 to propagate the consequences of the assignment.
4. If a domain becomes empty, reject the choice and restore the previous domains.
5. Otherwise, continue the recursive search.
6. Display the final channel assignment or report failure.

Part C: Compare the techniques

Run the following approaches on the same CSP:

- Basic backtracking.
- Backtracking with forward checking.
- Backtracking with MAC.

Report the number of candidate assignments attempted by each approach.

Analysis questions

- Why does AC-3 recheck neighboring arcs after a domain changes?
- Does successful AC-3 execution guarantee that a complete solution exists?
- How can MAC discover inconsistencies that forward checking may miss?
- Why might MAC perform more computation per assignment than forward checking?

Learning outcome: *Students will understand arc consistency, queue-based propagation, and the integration of propagation with recursive search.*

Task 5: Network Configuration Using a Tree-Structured CSP

A university has five wireless access points arranged along a corridor. Each access point must use a channel different from the access points directly connected to it.

Because the network connections form a simple chain, the constraint graph is a tree. Your task is to use the tree-structured CSP algorithm to solve the configuration problem.

Given data

The access points and their initial channel domains are:

Access point	Available channels
A	1
B	12
C	12
D	23
E	3

The network topology is:

A — B — C — D — E

The constraint between every connected pair is that their channel numbers must differ.

Requirements

1. Represent each access point as a variable and its available channels as its domain.
2. Construct the constraint graph and verify that it is connected and contains no cycles.
3. Choose a root and order the variables from root to leaves.
4. Process the variables in reverse order, removing parent-domain values that have no compatible value in the corresponding child domain.
5. Report failure if a parent domain becomes empty.
6. After the backward-processing phase, assign a value to the root.
7. Assign values to the remaining variables from root to leaves, ensuring that each assigned value satisfies the constraint with its parent.
8. Display the resulting channel configuration and verify every constraint.
9. Count the number of domain values removed during the backward-processing phase.

Analysis questions

- Why does the algorithm process variables in reverse order before assigning values?
- Why must the constraint graph be a tree for the stated tree-structured algorithm to apply directly?
- How does exploiting the graph structure differ from using general backtracking search?

Learning outcome: Students will learn how a CSP's graph structure can be exploited to solve certain problems more efficiently.

Task 6: Crack the Secret Code: SEND + MORE = MONEY

A secret communication system encodes digits as letters. Each letter represents a unique digit, and the following arithmetic equation must be satisfied:

SEND

+ MORE

MONEY

Your task is to formulate this cryptarithmetic puzzle as a CSP and implement a backtracking-based solver.

Rules

1. Each letter represents a digit from 0 to 9.
2. Different letters must represent different digits.
3. The same letter must represent the same digit wherever it appears.
4. The leading letters S and M cannot be zero.
5. The arithmetic equation must hold exactly.

Part A: Formulate the CSP

1. Identify the eight letter variables: S, E, N, D, M, O, R, Y.
2. Define their digit domains, accounting for the leading-zero restriction.
3. Identify the all-different constraint.
4. Explain how the arithmetic equation restricts valid assignments.
5. Define the carry variables and write the column-wise arithmetic equations.

Use the following carry equations in your implementation or reasoning:

- Units: D + E = Y + 10C1
- Tens: N + R + C1 = E + 10C2
- Hundreds: E + O + C2 = N + 10C3
- Thousands: S + M + C3 = O + 10C4

- Ten-thousands: C4 = M

Part B: Implement backtracking

1. Maintain a dictionary of letter-to-digit assignments.
2. Maintain a set of digits already used.
3. Select an unassigned letter and try available digits.
4. Reject a digit if it is already assigned to another letter.
5. Reject zero for S and M.
6. Recursively assign digits to the remaining letters.
7. Undo the assignment and release the digit when a branch fails.
8. At a complete assignment, verify that SEND + MORE = MONEY.
9. Display the digit assigned to each letter and the resulting arithmetic equation.

Part C: Improve the solver

Implement at least one improvement:

- Option 1: Check the column-wise carry equations as soon as the relevant digits and carries have been assigned.
- Option 2: Use MRV to select the next letter based on the number of legal digits remaining.
- Option 3: Use forward checking to remove digits that are no longer legal for other letters.

Compare the improved version with your baseline solver using the number of candidate digit assignments attempted.

Analysis questions

- Why is checking only whether digits are unique insufficient to establish a solution?
- Why can the leading letters not be assigned zero?
- How can checking a column equation early prevent unnecessary search?
- Why can a general binary-CSP solver not automatically enforce every arithmetic constraint in this puzzle?

Learning outcome: *Students will apply CSP modeling, backtracking, higher-order constraints, and constraint checking to a problem that combines arithmetic and logical reasoning.*

****